Project Name: Meeting Police

Team Members:

1. Vio Gustian Nur Alamsyah
2. Nur Aji Ananta

Problem (The "Why")

- **Current Issue:**

  Meetings usually have a predefined agenda containing important topics that need to be discussed. However, during a meeting, discussions can easily move between different subjects, causing some agenda items to be unintentionally skipped.

  Most existing meeting tools focus on recording meetings, generating transcripts, or producing summaries after the meeting. They do not actively help participants determine whether all required topics have been discussed before the meeting ends.

- **Impact:**

  Missing important agenda items can result in:

  - Important decisions being postponed.
  - Incomplete meeting outcomes.
  - Additional follow-up meetings.
  - Miscommunication between participants.
  - Reduced meeting efficiency and productivity.

  The problem is therefore not simply a lack of meeting documentation, but the lack of real-time awareness of meeting agenda coverage.

Proposed Solution

- **Our Approach:**
  We propose an AI-Powered Real-Time Meeting Coverage Assistant, a web-based application that monitors meeting discussions and compares them against a predefined agenda.

  Before the meeting starts, users define the topics that must be discussed.

  During the meeting, the application receives the meeting audio and uses an AI-powered Speech-to-Text service to convert speech into text. The generated transcript is then analyzed using AI-based semantic analysis to determine whether the conversation is related to any predefined agenda item.

  When an agenda item is identified as sufficiently discussed, its status is automatically updated.

  The system continuously monitors the remaining agenda items. If the meeting is approaching its scheduled end while important topics remain undiscussed, the application displays a real-time warning to the participants.

  Core Workflow:
  1. Users create a meeting and define the required agenda items.
  2. The meeting starts and the application receives the meeting audio.
  3. Speech is converted into text using a speech-to-text mechanism.
  4. The system analyzes the transcript and matches the discussion against the predefined agenda.
  5. Detected agenda items are automatically marked as **Discussed**.
  6. The application continuously monitors the meeting progress.
  7. If the meeting is approaching its scheduled end while important topics remain undiscussed, the system displays a warning.
  8. After the meeting, users can review which topics were discussed and which were missed.

- **Key Value:**
  The key value of this application is that it changes the role of meeting

software from being primarily **reactive** to being **proactive**.

Instead of simply answering: "What happened during the meeting?"
the system helps answer: "Have we covered everything we needed to discuss?"

This allows the team to identify missed agenda items before the meeting ends, reducing the possibility of incomplete meeting outcomes.

Core Features

- **Feature 1: Meeting Agenda & Priority Management**
  Users can create a meeting and define the topics that must be discussed. Each agenda item can contain:
  1. Topic name
  2. Description
  3. Priority
  4. Expected discussion keywords or phrases

  The agenda becomes the reference point used by the system to monitor meeting coverage.

- **Feature 2: AI-Powered Real-Time Discussion Tracking**
  The system captures meeting audio and sends it to an integrated Speech-to-Text service.

  The generated transcript is processed in manageable chunks rather than sending the entire meeting transcript for every request.

  The system then uses semantic analysis to determine whether the transcript is related to a predefined agenda item.

  When a topic is detected, its status can change from **Pending → Discussed**

  The React dashboard receives the status update in real time through ASP.NET Core SignalR.

- **Feature 3: Missing Topic Detection & Real-Time Warning**

  The system continuously checks the meeting's agenda coverage.

  When the meeting approaches its scheduled end, the system checks for important agenda items that are still marked as pending.

  For example:

  “Meeting ending soon. 2 important topics have not been discussed.”

  The application highlights the remaining topics so participants can address them before ending the meeting.

High-Level Architecture & Quality Assurance

- **Architecture Pattern:** Layered Architecture
- **Frontend:** React
- **Backend:** ASP.NET Core Web API using .NET SDK 8
- **Database:** PostgreSQL
- **Design Patterns Used:**
  - Dependency Injection
  - Repository Pattern
  - Service Layer
  - Strategy Pattern where appropriate for discussion/topic matching
  - DTO Pattern for API communication
- **Real-Time Communication:** ASP.NET Core SignalR
- **AI / Speech Processing:** AI-powered Speech-to-Text API, Large Language Model API for semantic topic detection
- **Code Quality Goal:** We commit to maintaining a green Jenkins pipeline, writing unit tests to be tracked via Cobertura, and keeping Qodana static analysis warnings below 10 (Target: 0).

Identified Challenges & Risks

- **Risk 1: Real-Time Speech-to-Text Processing**

Converting meeting audio into text with sufficiently low latency may be challenging, especially when dealing with continuous audio.

Mitigation:

Use an existing Speech-to-Text API instead of developing a speech recognition model from scratch.

For the initial prototype, audio can be processed in short chunks to simplify implementation and control API usage.

- **Risk 2: AI Topic Detection Accuracy**

  A simple keyword search may not be sufficient to determine whether an agenda item has actually been discussed.

  For example: Agenda: "Discuss the Q4 project budget"

  The meeting might say: "We need to increase the infrastructure allocation by $10,000."

  The system needs to understand the semantic relationship rather than relying only on exact keyword matching.

  Mitigation:

  Use an AI/LLM-based semantic analysis layer to classify the relationship between transcript chunks and agenda items.

  A confidence score can also be used to prevent the system from automatically marking topics as discussed when the detection confidence is too low.

- **Risk 3: AI API Latency and Availability**

  External AI services may introduce latency or become temporarily unavailable.

  Mitigation**:**

  The AI integration will be isolated behind an abstraction/interface in the Infrastructure layer. The system can also fall back to basic keyword

matching or mark the transcript as requiring manual verification if the AI service is unavailable.

- **Risk 4: AI Cost and Excessive API Requests**

  Sending every individual sentence to an AI service could result in excessive API usage, higher costs, and unnecessary latency.

  Mitigation**:**

  The system will process transcript data in controlled chunks and only send relevant transcript segments for semantic analysis.

  A lightweight pre-filter can also be applied before calling the LLM.

- **Risk 5: False Positive / False Negative Detection**

  AI may incorrectly determine that a topic has or has not been discussed.

  Mitigation:

  The system will use confidence thresholds and allow users to manually change the agenda status when necessary.

  The AI should assist participants rather than completely replace human judgment.

- **Risk 6: Limited Two-Week Development Time**

  A fully production-ready AI meeting assistant would require significantly more development time.

  Mitigation:

The project scope will focus strictly on the core problem-solving workflow:

